From Black-Box to White-Box Understanding of an Autoresearch Loop: A Pilot Study on Instruction-Driven LLM Behavior

Christian Block

July 2026

Abstract

Compound, LLM-driven research systems – agents that read an instruction file, form hypotheses, run experiments, and decide what to do next – are increasingly used as a substitute for human researchers on narrow technical questions. Yet it is usually unclear *why* such a system behaves the way it does: if we change the instructions, the output changes, but we rarely know which part of the instructions caused which part of the change. This report describes a pilot study that treats the instruction file, called `program.md`, the same way AutoML treats a hyperparameter vector: as a small, structured configuration space whose causal effect on system behavior can be measured with standard design-of-experiments tools. We decompose `program.md` into five interpretable axes, generate a resolution-III ablation design (Plackett–Burman, 8 configurations), execute each configuration against a real machine learning task (tuning Random Forest and Gradient Boosting classifiers on a benchmark dataset), encode the resulting process traces into behavioral features, fit an interpretable surrogate model relating configuration to behavior, and validate the surrogate causally by predicting the behavior of a held-out configuration before running it. The pipeline reproduces a known ground truth cleanly (an axis that instructs verbosity explains 74% of the variance in output length) and surfaces one non-obvious effect (an axis governing explore-vs-exploit search strategy explains 84% of the variance in achieved model accuracy). A held-out causal validation run confirms the surrogate's prediction for one behavioral feature almost exactly and misses on another by a small, informative margin. We report this honestly as a pilot: due to practical constraints, one methodological compromise (the "LLM" role was played by the author rather than by an independent, freshly-initialized language model call) limits how much weight the specific numbers should carry, and we describe exactly what a full-scale follow-up would need to change.

1 Introduction

1.1 Motivation

Large language models are now used not just to answer questions but to run small autonomous research loops: given a natural-language instruction file and a tool to execute experiments, the model repeatedly proposes an experiment, runs it, looks at the result, and decides what to try next. This pattern – sometimes called an *autoresearch loop* – can replace a human iterating on a machine learning problem: tuning a model's hyperparameters, comparing algorithms, or searching for a good configuration.

The appeal of such a system is obvious, but so is a specific worry. The system's entire behavior is steered by one artifact: a text file of instructions, here called `program.md`. Two research groups running "the same" autoresearch loop on "the same" problem, but with slightly different instruction files, can get meaningfully different research processes and different final results. In practice, people notice this (*"if I phrase the stopping rule differently, the agent runs way fewer experiments"*) but rarely go further than noticing it. The relationship between an instruction and a behavior is treated as a black box: we can observe that changing the instruction changes the output, but we cannot say which part of the instruction

is responsible, nor by how much.

This is unsatisfying for the same reason a black-box machine learning model is unsatisfying to a scientist: correlation without a causal, quantitative account does not let us predict what will happen under a change we have not yet tried, and it does not let us explain the system's behavior to somebody else in a way they can act on.

1.2 Research question and goal

The research question this project addresses is methodological rather than purely empirical:

> *Can a compositional instruction file be decomposed into a small number of interpretable components whose individual causal contribution to an LLM-driven research loop's behavior can be measured and predicted – using the same tools that AutoML researchers already use to explain the effect of hyperparameters on model performance?*

We are deliberately borrowing methodology rather than inventing new theory. The AutoML literature has spent over a decade developing tools for exactly this kind of problem in a different setting: given an algorithm with many hyperparameters, decide which hyperparameters actually matter, by how much, and whether they interact. Techniques such as ablation analysis [1], functional ANOVA over hyperparameter importance [2], and fractional factorial design of experiments [3] were built for this. Our claim is that an instruction file like `program.md` is structurally the same kind of object as a hyperparameter vector – a set of independent, discrete choices that jointly determine a system's behavior – and that these tools transfer directly.

To keep the space of possible research behaviors small enough to study exhaustively, we restrict the domain to a single, well-understood machine learning task: tuning Random Forest and Gradient Boosting classifiers on a fixed tabular dataset. This is deliberately a "boring" and well-studied domain; the interesting question is not whether the agent can tune these models well, but whether we can explain *why* it tunes them the way it does under a given instruction file.

1.3 Contributions of this pilot

This report documents one complete, executed pass through a six-step research agenda (Section 3), using real machine learning experiments throughout – no part of the numerical results in this report is simulated. Concretely, we:

1. Decompose `program.md` into five two-level axes and build a template-based generator that renders any of the 32 possible instruction files from a five-bit configuration vector (Section 3.2).
2. Generate an 8-run Plackett–Burman ablation design over these axes and execute each configuration, with repeats, against a real scikit-learn experiment executor (Sections 3.3–3.4).
3. Parse the resulting structured process traces into a small set of quantitative behavioral features (Section 3.5).
4. Fit an interpretable surrogate model (linear main effects plus a shallow decision tree) that explains which configuration axis drives which behavioral feature, and by how much (Section 3.6).
5. Use the surrogate to predict the behavior of a held-out, previously unseen configuration *before* running it, then run it for real and compare (Section 3.7).
6. Report honestly on the scope and limitations of this pilot, in particular a methodological compromise in how the "LLM" decisions were produced, and lay out what a full-scale replication needs to change (Section 6).

The remainder of this report is structured as a standard experimental methodology write-up: Section 2 gives brief background on the AutoML techniques we borrow; Section 3 describes each of the six steps in detail; Section 4 describes the experimental setup precisely enough to reproduce; Section 5 presents the results; Section 6 discusses limitations; Section 7 concludes.

2 Background

Three ideas from the AutoML and experimental-design literature underlie this project. Readers already familiar with design of experiments can skip to Section 3.

Hyperparameter importance analysis. When an algorithm has several hyperparameters, a natural question is which ones actually matter for performance, and how much of the algorithm's overall performance variance each one is responsible for. Functional ANOVA [2] answers this by decomposing the variance of an outcome (e.g. validation accuracy) into a sum of contributions attributable to each hyperparameter individually, plus contributions from interactions between pairs (or larger groups) of hyperparameters. The appeal is that the output is a small set of interpretable numbers – "hyperparameter A explains 60% of the variance, hyperparameter B explains 5%" – rather than a black-box performance surface. We reuse exactly this idea, but apply it to axes of an instruction file instead of axes of a hyperparameter vector.

Ablation analysis. Directly measuring the effect of every hyperparameter, including every combination of hyperparameters, is combinatorially expensive: with k two-level hyperparameters there are 2^k possible combinations. Ablation analysis [1] avoids this by tracing the performance difference between two specific configurations along a path of single-hyperparameter changes, attributing the total difference to each change along the path. A related but distinct idea, which we use here, is fractional factorial *design of experiments*: instead of running all 2^k combinations, one runs a carefully chosen subset that still permits every individual ("main") effect to be estimated without bias, at the cost of not being able to separate two-way interaction effects from each other. The classical Plackett–Burman construction [3] produces such a subset in multiples of four runs (e.g. 8 runs for up to 7 two-level factors) and is standard practice in industrial experimentation and, increasingly, in algorithm configuration research.

Surrogate models. A surrogate model is simply an interpretable model – a shallow decision tree, a linear regression, or an ANOVA decomposition – fit on a table of (configuration, outcome) pairs, standing in for the true, usually much more complex, relationship between a configuration and a system's behavior. The point of a surrogate is not predictive accuracy for its own sake, but interpretability: if the surrogate can be inspected (e.g. "a one-line-item change in axis X increases the outcome by δ"), and if that inspected relationship is later confirmed by directly intervening (changing X and observing the predicted δ), we have moved from a correlational account to a causal one. This last step – intervention rather than mere observation – is what distinguishes the white-box account this project aims for from a black-box one.

3 Methodology

The overall pipeline has six steps, matching a pre-registered research agenda. Steps 2 through 6 were fully executed for this pilot; Step 1 was intentionally deferred (see Section 6). Figure-free description follows, since every step is a data transformation from one table or file format to the next; the actual flow is:

`program.md axes` → `ablation design` → `executed runs`
→ `behavioral table` → `surrogate model` → `causal check`

3.1 Step 1: Human baseline (not executed in this pilot)

The original agenda calls for a human researcher to solve one concrete sub-problem in the domain (e.g. "does the feature subsampling ratio affect generalization of a Random Forest on dataset X") while thinking aloud, with the resulting recording transcribed and segmented into a sequence of actions using a small taxonomy: *hypothesize, design experiment, execute, observe, interpret, decide next step*. This reference trace would then serve as a benchmark against which every automated run could be compared (e.g. "how similar is the agent's search process to how a human would have approached this?"). Producing this trace requires a person to actually perform the task under recording; it cannot be substituted computationally, so it was excluded from this pilot by deliberate agreement and is the first item on the follow-up list (Section 6). Importantly, the action taxonomy used for the human trace was still adopted for every other step below, so that once a human trace exists, no other part of the pipeline needs to change to make use of it.

3.2 Step 2: `program.md` as a configuration space

The first executed step turns an unstructured instruction file into a small, structured configuration vector, mirroring how a hyperparameter vector turns an algorithm into something whose behavior can be studied systematically. We identified five two-level axes, chosen because each corresponds to an instructional choice that plausibly affects behavior and that appears, in some form, in real instruction files:

- **M – Metric.** Is the evaluation criterion left vague ("evaluate model quality in whatever way you judge appropriate"), or made precise and stability-aware ("report mean 5-fold cross-validated accuracy *and* its standard deviation; use the standard deviation only to break ties")?
- **B – Breadth.** Must the agent stay within a single model family for the whole session (Random Forest *or* Gradient Boosting, not both), or is it free to compare both side by side?
- **S – Stopping rule.** Is the experiment budget fixed at exactly 3 experiments, or adaptive (continue up to 8 experiments, stopping as soon as one experiment fails to improve the metric by more than 0.002)?
- **O – Output format.** Should responses be terse and numeric, or fully reasoned prose explaining hypotheses and interpretations?
- **E – Emphasis.** Should the agent prioritize breadth (try several different configurations before refining any one of them), or depth (immediately refine the first promising configuration with nearby variations)?

Each axis has exactly two levels, coded 0 and 1. A configuration is therefore a five-bit vector, e.g. $(M, B, S, O, E) = (1, 0, 0, 1, 1)$. Practically, we do not hand-write each variant of `program.md`. Instead, a single Jinja2 template with placeholders for each axis's instructional text is filled in by a small Python script given a configuration vector, guaranteeing that any two rendered instruction files differ *only* in the axes that were intentionally varied – an essential property for any later step that wants to attribute a behavioral difference to a specific axis.

3.3 Step 3: Ablation instead of random sampling

With five two-level axes, a full factorial design requires $2^5 = 32$ configurations, and because the loop itself is stochastic, each configuration ideally needs several repeats to separate the effect of the configuration from ordinary run-to-run noise. Running all 32 configurations with several repeats each was not affordable within the scope of this pilot, so we instead used a **Plackett–Burman design**: a classical fractional factorial construction that selects 8 of the 32 configurations such that the main effect of every one of the five axes can still be estimated without being confounded with any other main effect. The cost of this efficiency is that two-way interactions between axes (e.g. "does the effect of the emphasis axis

depend on the breadth axis?") cannot be separated from each other; this is a deliberate trade-off known as a resolution-III design, and we return to its consequences in Section 3.7.

The 8-configuration design was generated by a small script implementing the standard cyclic construction of the design (a base row of $+, +, +, -, +, -, -$, cyclically shifted seven times, plus one row of all $-$). Table 1 lists the resulting 8 configurations.

Table 1: The 8-run Plackett–Burman ablation design over the five axes. 0/1 correspond to the two levels of each axis described in Section 3.2.

Config #	M	B	S	O	E
0	1	1	1	0	1
1	0	1	1	1	0
2	0	0	1	1	1
3	1	0	0	1	1
4	0	1	0	0	1
5	1	0	1	0	0
6	1	1	0	1	0
7	0	0	0	0	0

Each of these 8 configurations was executed twice, with two different random seeds (0 and 1), giving 16 runs in total for the main ablation study. The agenda recommends 3–5 repeats per configuration to properly separate configuration effects from stochastic noise; 2 repeats were used here to keep the pilot's scope tractable, and this is listed as a concrete limitation.

3.4 The autoresearch loop and its executor

Each run consists of a sequence of steps in which the agent proposes an experiment, an executor actually runs it, and the agent interprets the result and decides whether to continue. Two design choices are worth making explicit, since they determine what "running the loop" means concretely in this pilot.

The task and the dataset. Every run addresses the same research question: how does the choice of hyperparameters affect the generalization accuracy of Random Forest and Gradient Boosting classifiers on the Breast Cancer Wisconsin diagnostic dataset (a standard, small, binary-classification benchmark bundled with scikit-learn)? The dataset is loaded once and split 70/30 into a training and validation set with a fixed random seed, so every run across every configuration operates on exactly the same data split; only the instruction file and the loop's own randomness vary between runs.

The executor tool. At each step, the agent may request one experiment by specifying a model family (Random Forest or Gradient Boosting) and a dictionary of hyperparameters. The executor – ordinary Python code, not an LLM – actually fits the requested model, computes 5-fold cross-validated accuracy and its standard deviation on the training split, and computes accuracy on the held-out validation split. It returns these real numbers to the agent. This is the one part of the loop that is deliberately *not* a language-model call: the numbers reported back to the agent are always genuine measurements from a real, executed scikit-learn model, never invented or approximated.

The agent's decisions. At each step, the agent produces one of two structured JSON actions: `propose` (a hypothesis plus a concrete model and hyperparameters to try) or `interpret` (an interpretation of the last result plus a decision to continue or stop, with a final recommendation if stopping). Every action is logged with a timestamp-equivalent step index, so that a run's full history is a simple, machine-readable sequence of typed entries – exactly analogous to the action taxonomy planned for the human baseline in Step 1.

3.5 Step 4: Behavioral encoding

Because every run already produces structured JSON rather than free text, turning a raw transcript into a row of a feature table is a mechanical parsing step rather than a task requiring natural-language classification. For each run we compute five behavioral features:

- **n_experiments** – how many real experiments were executed before the agent stopped.
- **n_distinct_models** – how many of the two model families (Random Forest, Gradient Boosting) were actually tried, a direct measure of how much the agent exploited its freedom to switch families.
- **breadth_spread** – the range (maximum minus minimum) of the `n_estimators` hyperparameter values tried across the run's experiments, used as a simple proxy for how widely the agent searched the hyperparameter space.
- **best_cv_accuracy** – the best cross-validated accuracy obtained among all experiments in the run: the "product" quality of the research.
- **mean_proposal_chars** – the average character length of the agent's stated hypotheses, a crude proxy for verbosity.

The output of this step is a single table with one row per run, holding the five-bit configuration vector, a repeat/seed identifier, and the five behavioral features above – ready to be modeled.

3.6 Step 5: Surrogate model

For each behavioral feature y, we fit a linear model $y \approx \beta_0 + \sum_i \beta_i x_i$, where each $x_i \in \{-1, +1\}$ codes one of the five axes. Because the Plackett–Burman design is orthogonal (each pair of coded axis-columns is uncorrelated across the 8 configurations), the five coefficients β_i can be estimated independently of each other, and – crucially for interpretability – the fraction of the model's total explained variance attributable to axis i is simply β_i^2 divided by the sum of all β_j^2. This gives a direct, per-axis "variance share": a single number, between 0% and 100%, stating how much of a given behavior is explained by a given axis, holding the resolution-III caveat about interactions in mind. As a non-parametric cross-check, we additionally fit a shallow (depth 3) decision tree for each behavioral feature and inspect whether it splits on the same axes the linear model identifies as important. Before fitting either model, the two repeats (seeds) for each configuration are averaged, so that the model is fit on 8 configuration-level rows rather than 16 run-level rows, matching the resolution of the ablation design itself.

3.7 Step 6: Causal validation

A surrogate model that only fits the data it was trained on is still, in an important sense, a black box – it describes a correlation, not a causal mechanism. The final step turns the surrogate into a testable, falsifiable claim. We selected one configuration, $(M, B, S, O, E) = (0, 0, 0, 0, 1)$, that was *not* among the 8 configurations in the original ablation design (Table 1), used the fitted surrogate to predict two of its behavioral features *before running it*, then rendered its `program.md`, executed it twice with two entirely new random seeds (2 and 3, never used elsewhere in this pilot), and compared the predicted values to what was actually observed. This is the step that lets us say more than "the surrogate correlates with the data"; it lets us say whether the surrogate's implied causal story – "if you set the axes this way, you will get approximately this behavior" – survives an actual intervention.

4 Experimental Setup

Dataset. The Breast Cancer Wisconsin diagnostic dataset, as bundled with `scikit-learn` (569 samples, 30 numeric features, binary classification). A fixed 70/30 train/validation split (random seed 0) was

used identically across every run in every configuration.

Models. scikit-learn's `RandomForestClassifier` and `GradientBoostingClassifier`. The agent could vary `n_estimators`, `max_depth`, `min_samples_leaf`, and `max_features` for Random Forest, and `n_estimators`, `max_depth`, `learning_rate`, and `subsample` for Gradient Boosting.

Evaluation metric reported by the executor. 5-fold cross-validated accuracy (mean and standard deviation) on the training split, plus a single accuracy measurement on the held-out validation split, for every experiment regardless of what the active `program.md` configuration emphasized.

Scale of the pilot. 8 ablation configurations $\times$ 2 repeats = 16 runs, plus 1 held-out configuration $\times$ 2 repeats = 2 further runs for causal validation, for a total of 18 executed runs and 46 individual model-fitting experiments across all runs combined.

Software. Python 3.12, scikit-learn, statsmodels (for the linear surrogate and significance tests), and a custom Jinja2-based template renderer for `program.md` generation. All code, generated instruction files, and raw run transcripts are stored alongside this report for full reproducibility.

5 Results

5.1 Surrogate model: which axis explains which behavior

Table 2 summarizes, for each of the five behavioral features, the axis with the largest variance share, the size of that share, the direction of the effect, and the linear model's overall R^2 on the 8 configuration-level data points.

Table 2: Surrogate model summary: dominant axis per behavioral feature.

Behavioral feature	Dominant axis	Variance share	Direction of effect	R^2
mean_proposal_chars (verbosity)	O	74%	verbose $\rightarrow$ roughly 2–4$\times$ longer proposals	0.96
n_distinct_models (models tried)	B, O, E (tied)	33% each	broad$\uparrow$, verbose$\uparrow$, exploit-first$\downarrow$	1.00
best_cv_accuracy (product quality)	E	84%	exploit-first $\rightarrow$ +0.0025 mean accuracy	1.00
n_experiments	S, E (tied)	50% each	adaptive stopping$\downarrow$, exploit-first$\uparrow$	0.67
breadth_spread (search width)	E	49%	exploit-first $\rightarrow$ wider spread	0.83

Two results deserve a closer look.

The output-format axis (O) explaining verbosity is a sanity check, and it passes. Axis O directly instructs the agent to be either terse or fully reasoned. That it explains 74% of the variance in proposal length, with a statistically significant coefficient ($p = 0.027$), is close to a tautology – but it is an important one. It confirms that the entire pipeline, from instruction template through execution, JSON logging, feature parsing, and linear model fitting, correctly recovers a known ground truth. Had this check failed (say, if some unrelated axis had appeared to dominate verbosity instead), it would have indicated a bug somewhere in the pipeline rather than a genuine surprising finding. Passing this check is a necessary, though not sufficient, condition for trusting the less obvious results below.

The emphasis axis (E) explaining achieved accuracy is the more interesting, non-obvious result. Axis E does not mention accuracy at all – it only instructs the agent to prioritize either breadth-first

exploration or immediate exploitation of a promising result. Yet it explains 84% of the (admittedly small) variance in the best cross-validated accuracy achieved across the 8 configurations, with the exploit-first strategy outperforming explore-first by about 0.0025 (roughly a quarter of an accuracy percentage point) on average, and a highly significant coefficient ($p < 0.001$). In the small hyperparameter neighborhoods explored in this pilot, immediately refining a promising configuration tended to beat spreading the same small experiment budget across more dissimilar configurations. This is plausible on its face – with only 2–4 experiments per run, a strategy that commits early to refining a good region has an advantage over one that "wastes" an experiment exploring a probably-worse region – but it is exactly the kind of instruction-level effect that would otherwise be invisible without this pipeline.

5.2 Causal validation

The held-out configuration $(M, B, S, O, E) = (0, 0, 0, 0, 1)$ – narrow model family, an unspecified evaluation metric, a fixed budget of exactly 3 experiments, terse output, and exploit-first emphasis – was not part of the 8-configuration ablation design. Table 3 compares the surrogate's predictions, made before this configuration was ever executed, to what was actually observed across two fresh runs.

Table 3: Causal validation: predicted versus observed behavior for a held-out configuration.

Behavioral feature	Predicted (before running)	Observed (after running)	Discrepancy
n_experiments	3.25	3.00	-0.25
best_cv_accuracy	0.9648	0.9610	-0.0038

The prediction for `n_experiments` is essentially exact: the surrogate correctly identified that this configuration's fixed-budget stopping rule (S = 0) is the dominant, near-deterministic driver of how many experiments get run, and predicted a value within a quarter of an experiment of what was actually observed. The prediction for `best_cv_accuracy` is off by about 0.0038, or roughly 0.4 accuracy points, in the direction of overestimating how much the exploit-first strategy (E = 1) helps. Because the ablation design used in Step 5 is resolution III, it cannot separate axis E's effect from possible interactions between E and the other axes (most plausibly B, the model-family breadth axis, or S, the stopping rule). The size and direction of this miss is consistent with E's true effect on accuracy depending on which other axis levels are present, rather than being a fixed additive constant as the linear surrogate assumes. Read in the spirit of the original agenda, this is not a failure of the method: a wrong-but-informative prediction that points to a specific, testable gap (an unmodeled interaction) is exactly what distinguishes a genuinely causal validation step from one that either always confirms or gives up.

6 Limitations

We list every limitation that materially affects how much weight the numbers in Section 5 should carry, in order of importance.

1. The agent's decisions were produced by the author, not by an independent language model call. This is the most significant limitation of this pilot. Producing genuinely independent, fresh-context decisions for 18 runs required a way to make repeated calls to a language model that had no memory of anything beyond the rendered `program.md` for that specific run. No such access was available in the environment this pilot was executed in. As a substitute, the author personally played the role of the research agent for every run, reading each rendered `program.md` and trying to follow only what it said. This is a real methodological confound: the author already knew, throughout, that this was a controlled ablation study and roughly what each axis was designed to test, which a language model given only the rendered instruction file would not know. The surrogate model results in Section 5 should therefore be read as evidence that *the analysis pipeline works end-to-end and can surface plausible,*

internally consistent effects, not as a validated claim about how an independent large language model actually behaves under these instructions. The most important next step for this project is exactly this: re-running the same 8-configuration design with an API-driven, independent language model call per run, and checking whether the effects reported here replicate.

2. No human baseline exists yet. Step 1 of the original agenda – a recorded, transcribed, and segmented human trace of solving the same kind of problem – was not produced. Consequently, one of the agenda's own planned behavioral features, the process-similarity between an automated run and a human's actual research process, does not yet exist. The action taxonomy the automated loop already logs against was deliberately chosen to match the taxonomy planned for the human trace, so this feature can be added later without re-running any of the experiments already done.

3. Small sample size. Two repeats per configuration were used, rather than the 3–5 recommended by the agenda to properly separate a configuration's true effect from ordinary stochastic run-to-run variation. With only 8 configurations, the design is resolution III, meaning that no two-way interaction between axes can be estimated; the causal validation result in Section 5 is itself evidence that at least one such interaction (most likely involving the emphasis axis E) is not negligible.

4. Narrow domain. All experiments concern a single, small, tabular dataset (Breast Cancer Wisconsin) and a modest hyperparameter menu for two model families. Findings here are scoped to this setting; generalizing to "random forests and gradient boosting" as a domain more broadly would require repeating the design over multiple datasets.

7 Conclusion and Future Work

This pilot executed five of the six steps of a research agenda aimed at turning the relationship between an LLM-driven research loop's instruction file and its behavior from a black box into a white box, using tools borrowed directly from AutoML hyperparameter-importance analysis: a structured configuration space, a fractional factorial ablation design, an interpretable surrogate model, and a causal validation step that tests the surrogate's predictions against a real, held-out intervention. Every number reported here came from real, executed machine learning experiments, not from a simulation standing in for them.

The pipeline correctly recovered a known ground-truth relationship (the output-format axis driving verbosity) and surfaced one plausible, non-obvious relationship (the explore/exploit emphasis axis driving achieved model accuracy), and its causal validation step produced exactly the kind of outcome such a step should be able to produce: one prediction confirmed almost exactly, and one informative miss that points precisely at what the next iteration of the design needs to add (an interaction term involving the emphasis axis).

The most important next steps, in priority order, are: (1) replace the author-as-agent substitution with genuine, independent, fresh-context language model calls and check whether the reported effects replicate; (2) record the Step 1 human baseline and add the process-similarity feature it enables; (3) move from a resolution-III to a resolution-IV or full-factorial design, with more repeats per configuration, to directly estimate the interaction effects this pilot's causal validation step flagged as likely present; and (4) repeat the study across more than one dataset to check how much of what was found here is specific to this particular benchmark.

References

[1] C. Fawcett and H. H. Hoos. *Analysing differences between algorithm configurations through ablation.* Journal of Heuristics, 22(4):431–458, 2016.

[2] F. Hutter, H. Hoos, and K. Leyton-Brown. *An efficient approach for assessing hyperparameter importance.* Proceedings of the 31st International Conference on Machine Learning (ICML), 2014.

[3] R. L. Plackett and J. P. Burman. *The design of optimum multifactorial experiments.* Biometrika, 33(4):305–325, 1946.